

EV Human Machine Interface

Application Design and Implementation

*Driver Dashboard, Media System, Diagnostics, Engineer Mode and Telemetry Framework
using Qt 6/QML, Raspberry Pi 5 and STM32F446RE*

Field	Detail
Application host	Raspberry Pi 5
Display	10.1" Waveshare capacitive touchscreen
UI framework	Qt 6 / Qt Quick (QML), C++17 backend
Toolchain	Qt Creator, CMake
Companion MCU	STM32F446RE VCU, UART @ 115200 baud
Document date	July 2026

Abstract

This report documents the design and implementation of a premium automotive Human Machine Interface (HMI) developed in Qt 6/QML on a Raspberry Pi 5. The system consumes real-time vehicle telemetry produced by an STM32F446RE-based Vehicle Control Unit (VCU), transmitted over UART at 115200 baud. Incoming telemetry is decoded by a key=value parser, propagated through a reactive C++ data model, and rendered on a 10.1-inch capacitive touchscreen via Qt Quick property bindings.

The application encompasses a driver-facing surface (Home dashboard, Music, Diagnostics, and Settings) and an engineer-facing diagnostic overlay accessible through a hidden long-press gesture. A runtime theme engine provides six named colour themes in both day and night modes. A software simulator replaces the physical serial link during development, exercising the full UI stack without hardware dependency. The media subsystem integrates local file playback, Spotify Web API, and Bluetooth audio through independent backend classes that share a common property contract with the QML layer.

This document is the companion volume to the STM32F446RE VCU report. It is written to be used as an engineering reference and is structured to mirror the format and rigour of the VCU companion report.

How to Read This Document

This report documents the current state of the driver- and engineer-facing HMI application that consumes the telemetry produced by the VCU described in the companion volume. It is written to be read front to back; each chapter is also self-contained enough to serve as a standalone reference.

- Chapter 1 is the lookup table: every reactive property, compile-time constant, and class allocation, with physical interpretation and update path.
- Chapter 2 presents the system from two altitudes — the driver's view (UI page structure) and the engineer's view (software block diagram).
- Chapter 3 provides a module-by-module brief on every hardware and software component the application depends on.
- Chapters 4 and 5 cover the UART communication protocol (frame format, parser pseudo code, watchdog) and the reactive data model (single-source-of-truth pattern, hardware/simulation abstraction, simulator tick loop).
- Chapters 6–9 walk the three driver-facing surfaces (Driver Mode pages, Theme Engine, Engineer Mode) and the logging subsystem.
- Chapters 10–13 present the design evaluation, limitations, future scope, and conclusion.
- Appendices A–D provide the full VehicleData property map, Music API contract, project folder structure, and references.

1 Nomenclature and Property Lookup Table

This chapter is the single source of truth for every symbol used in the backend and throughout this report. Each entry states what the property is in software, what physical quantity it represents, and which signal path triggers its update.

1.1 Reactive Properties (Q_PROPERTY)

Every entry in Table 1.1 is a Qt Q_PROPERTY on VehicleData with a getter, a setter, and a *Changed() notify signal. A QML element bound to any of these is re-evaluated automatically the instant the notify signal fires; no polling is used anywhere in the UI layer.

Property	Represents	Update Path
speed	Vehicle speed (km/h)	SPD= token → setSpeed() → speedChanged()
rpm	Motor RPM	RPM= token → setRpm() → rpmChanged()
batteryPercent	Battery state of charge (0–100 %)	BAT= token → setBatteryPercent()
motorTemp / batteryTemp	Subsystem temperatures (°C)	MT= / BT= → WarningManager threshold check
controllerTemp	Controller temperature (°C)	CT= token → setControllerTemp()
range	Estimated remaining range (km)	RNG= token → setRange()
driveMode	Active throttle map: ECO / CITY / SPORT	MODE= token; mirrors VCU current_drive_mode
gearState	PRND gear state	GEAR= token; mirrors VCU gear_state FSM
motorPower	Instantaneous motor power (kW)	PWR= token (float)
packVoltage / packCurrent	Battery pack voltage (V) and current (A)	VOLT= / CURR= tokens
regenLevel	Regenerative braking level	REGEN= token
odometer / tripDistanceA / tripDistanceB	Cumulative and resettable trip distance	Integrated locally from speed; not transmitted
leftIndicator / rightIndicator	Turn-signal state	IND_L= / IND_R= tokens; mirrored to VehicleData flags
beamActive / hazardActive / brakeActive	Lighting flags	BEAM= / HAZ= / BRK= tokens
warningActive (×5)	Five independent warning flags	Raised by WarningManager against configured thresholds
communicationFault	True when no valid frame within timeout window	SerialManager watchdog timer
framesReceived / invalidFrames / checksumErrors	Running link-health counters	TelemetryParser; incremented per frame

Table 1.1: Reactive Q_PROPERTY declarations on VehicleData and their update paths.

1.2 Compile-Time Constants

Constant	Value	Meaning / Justification
UART_BAUD_RATE	115200	Matches STM32 USART2 configuration.
TELEMETRY_INTERVAL_MS	100	10 Hz update rate — above human visual threshold, well below UART capacity.
COMM_TIMEOUT_MS	1000	Ten missed frames before communicationFault is raised.
CONTRAST_DEFAULT	55 %	Default position of the elevation-contrast slider.
TOUCH_TARGET_BASE	56 px (pre-scale)	Multiplied by Theme.qml scale factor for every tap target.
MOTOR_TEMP_WARNING_C	55 °C	HMI warning threshold; above VCU derate point of 50 °C to warn ahead of firmware intervention.
MOTOR_TEMP_FAULT_C	70 °C	HMI fault threshold; above VCU cutoff of 65 °C.
BATTERY_LOW_WARNING_PCT	15 %	Below this, a low-battery warning banner is raised.

Table 1.2: Compile-time constants and engineering rationale.

1.3 Class Allocation

Class	Responsibility
VehicleData	Central reactive data model; single source of truth exposed to QML.
SerialManager	Owns the physical UART connection via QSerialPort in hardware mode.
TelemetryParser	Decodes key=value telemetry frames; writes typed values into VehicleData.
TelemetrySimulator	Generates synthetic telemetry for development and testing without hardware.
TelemetryLogger	Persists telemetry frames and fault events to timestamped CSV files.
WarningManager	Evaluates thresholds (over-temperature, low battery, comms) and raises warning state.
LocalMusicPlayer	Wraps QMediaPlayer; exposes stable playback API to QML.
SpotifyAPIManager	Integrates Spotify via Qt Network / OAuth 2.0 behind the same media contract.
BluetoothManager / DummyBluetoothManager	Bluetooth device management with no-hardware fallback.

Table 1.3: Backend class allocation. Each class owns exactly one resource domain.

2 System Overview

2.1 The Driver's View — User Interface

The driver-facing application exposes four pages reachable from a persistent bottom navigation bar, plus one always-visible top status bar:

- Home — speedometer, digital RPM, battery panel, gear/drive-mode state, turn indicators, system-status icons, odometer/trip counters.
- Music — local playback and Spotify behind a tabbed interface, with synchronised lyrics and a Bluetooth/media-hub panel.
- Diagnostics — a simplified health overview: overall score, per-subsystem status, active warnings, and fault history.
- Settings — units, language, font, theme, brightness/contrast, volume, clock/date, and factory reset.
- Top status bar — connection health indicator, system time, active drive mode, and battery percentage. Persistent across all pages.

Engineer Mode is intentionally absent from this list. It is reachable through a long-press gesture on the Diagnostics page and is covered in Chapter 9.

2.2 The Engineer's View — Software Block Diagram

Telemetry originates at the STM32 VCU and arrives over UART at the Raspberry Pi 5, where SerialManager and TelemetryParser decode it into VehicleData. The QML UI layer subscribes through Qt's property-binding system and never communicates with the serial port directly. The layered architecture below captures these relationships:

```
STM32 VCU (Vehicle Sensors, PRND FSM, Drive Modes)
  ↓ UART / 115200 baud
SerialManager (QSerialPort)
  ↓ raw frame bytes
TelemetryParser (key=value tokenizer)
  ↓ typed values
VehicleData (Q_PROPERTY reactive model)
  ↓ Qt property bindings
WarningManager | TelemetryLogger
  ↓ QML property bindings
QML Pages (Home, Music, Diagnostics, Settings, Engineer Mode)
```

[Figure 2.1 — Software block diagram]

Figure 2.1: Software block diagram. Every UI screen reads from the same VehicleData instance. The data source (SerialManager or TelemetrySimulator) is selected once at startup and is transparent to the UI.

Three architectural decisions warrant explicit justification:

Single source of truth. Every screen reads from the same VehicleData instance, so no two views can ever disagree about the current speed or battery level.

Interchangeable data source. Because the UI depends only on VehicleData, the producer can be swapped between SerialManager and TelemetrySimulator behind a single IDataSource interface without modifying any QML file.

Domain ownership boundary. Vehicle telemetry is never added to any media backend class, and music metadata is never added to VehicleData. Each backend object owns exactly one domain of state.

3 Hardware and Software Modules

3.1 Raspberry Pi 5 — HMI Host

A quad-core Arm Cortex-A76 application processor running the Qt/QML application under a standard Linux userspace. It performs no real-time vehicle control; its only hard timing obligation is servicing the UART line fast enough that the 100 ms telemetry cadence does not back up. GPIO pins on the 40-pin header carry the UART interface directly to the STM32's USART2 (PA2/PA3).

3.2 Waveshare 10.1" Capacitive Touchscreen

A 10.1-inch capacitive touch panel connected via the Pi's display interface. Touch input is handled natively by Qt's event system. Layout is managed through a single Theme.qml scale factor multiplied into every spacing, radius, and touch-target constant (touchTarget: 56 × scale), which would allow the UI to be re-targeted to a different panel size without modifying individual page files.

3.3 Qt 6 / Qt Quick (QML)

The declarative frontend. Every screen is a tree of QML items bound to C++-exposed properties; the framework's property-binding engine removes the need for any manual re-render or polling logic in the UI layer. The application uses QML's singleton pattern for Colors.qml, Typography.qml, and Theme.qml, giving every component access to the complete design-token set without passing references down the component tree.

3.4 VehicleData

The architectural centrepiece of the backend (see Chapter 1 for the full property list). Every setter follows the same pattern: store the new value and emit the corresponding *Changed() signal only if the value actually differs. This single rule makes the entire UI reactive without polling and without redundant re-renders at the 10 Hz telemetry cadence.

3.5 SerialManager / TelemetryParser

SerialManager owns the physical UART connection via QSerialPort and emits raw lines as Qt signals. TelemetryParser decodes them. The two classes are kept separate: one owns the resource, the other owns the interpretation — the same discipline applied to ADC channels in the VCU firmware. SerialManager also runs the communication-fault watchdog timer that raises communicationFault in VehicleData if no valid frame arrives within COMM_TIMEOUT_MS.

3.6 TelemetrySimulator

A software stand-in for the STM32, driven by a PlayStation 3 controller or USB keyboard. It maintains an internal SimulationState structure that advances on a QTimer tick, modelling a thermally-evolving vehicle rather than emitting static or random values. Its internal tick loop and the IDataSource interface that makes it interchangeable with SerialManager are detailed in Chapter 5.

3.7 WarningManager

A pure C++ component holding threshold values for all monitored quantities — motor temperature, battery temperature, battery SoC, and communication health — and emitting warning-state changes into VehicleData whenever a threshold is crossed. Centralising thresholds here avoids the maintenance problem of hunting for a threshold change across multiple QML files.

3.8 LocalMusicPlayer / SpotifyAPIManager

LocalMusicPlayer wraps QMediaPlayer to expose local audio playback (track, artist, album art, progress, controls) to QML through a stable property contract. SpotifyAPIManager provides the same

contract for Spotify-sourced content through Qt Network and OAuth 2.0. Both follow the same property-naming conventions as VehicleData, so the Music page QML is agnostic about which backend is active.

3.9 BluetoothManager / DummyBluetoothManager

BluetoothManager manages Bluetooth audio device pairing, connection state, and active-audio routing using Qt Bluetooth. DummyBluetoothManager is a no-op stand-in used when Bluetooth hardware is unavailable on the development machine, keeping the remainder of the UI fully demonstrable. Its continued presence signals that Bluetooth hardware validation was not yet complete at the time of writing.

3.10 TelemetryLogger

Captures timestamped telemetry frames and fault records to CSV files on the local filesystem, supporting post-session analysis and field diagnostics. Covered in detail in Chapter 10.

4 Communication Framework

4.1 UART Link Parameters

Parameter	Value
Baud rate	115200
Data bits / parity / stop bits	8 / none / 1 (8N1)
Update interval	100 ms (10 Hz)
Communication timeout	1000 ms → communicationFault = true
Physical connection	Raspberry Pi 5 GPIO UART ↔ STM32 USART2 (PA2/PA3)

Table 4.1: UART link configuration.

4.2 Telemetry Frame Format

Each frame is a single newline-terminated line of comma-separated key=value tokens. An example frame:

```
SPD=54,RPM=2800,BAT=87,MT=41,BT=35,RNG=120,PWR=4.3,MODE=ECO,GEAR=D
```

The tagged format is a deliberate robustness choice: because each field is explicitly labelled, the parser can skip malformed or unrecognised tokens without corrupting the interpretation of remaining fields, and either side can add a new token without breaking the other. Fixed-position CSV would provide no such resilience.

Key	Field	Type	Unit / Domain
SPD	Vehicle speed	int	km/h
RPM	Motor RPM	int	RPM
BAT	Battery state of charge	int	%
MT / BT	Motor / battery temperature	int	°C
CT	Controller temperature	int	°C
RNG	Estimated range	int	km
PWR	Motor power	float	kW
MODE	Drive mode	string	ECO / CITY / SPORT
GEAR	Gear state	string	P / N / R / D
VOLT / CURR	Pack voltage / current	float	V / A
REGEN	Regen level	int	0–5

Table 4.2: Telemetry frame key definitions.

4.3 Pseudo Code — Frame Decode

The pseudo code below mirrors `TelemetryParser::parsePacket`, tokenising on commas and matching each key independently rather than assuming a fixed field order.

```
/* 1 // split the incoming line on ',' -- one token per field */
tokens = split(line, ',');

/* 2 // each token is KEY=VALUE; unknown or malformed tokens
   are skipped, not fatal */
for (token in tokens) {
    [key, value] = split(token, '=');
    if (!isRecognizedKey(key)) continue;
    if (!isWellFormed(value)) { invalidFrames++; continue; }

    /* 3 // dispatch by key -> matching VehicleData setter */
    switch (key) {
        case "SPD": vehicleData.setSpeed(toInt(value)); break;
        case "RPM": vehicleData.setRpm(toInt(value)); break;
        case "BAT": vehicleData.setBatteryPercent(toInt(value)); break;
        case "MT":  vehicleData.setMotorTemp(toInt(value)); break;
        case "PWR": vehicleData.setMotorPower(toFloat(value)); break;
        /* ... remaining keys follow the same pattern ... */
    }
}
framesReceived++;
lastFrameTick = now();
```

Listing 4.1: Key=value tokenising in `TelemetryParser::parsePacket`.

4.4 Pseudo Code — Reactive Property Update

Every `Q_PROPERTY` setter in `VehicleData` follows the same shape. This single pattern, applied uniformly, is the entire reason the UI layer never requires a `refresh()` call.

```
void VehicleData::setSpeed(int value) {
    /* 1 // no-op if value unchanged -- keeps re-render
       cost near zero at 10 Hz */
    if (m_speed == value) return;
    /* 2 // store, then notify */
    m_speed = value;
    emit speedChanged();
}
```

Listing 4.2: Store-then-notify setter pattern, repeated for every reactive property.

4.5 Pseudo Code — Communication Watchdog

```
/* runs on a 1000 ms QTimer tick inside SerialManager */
void SerialManager::checkTimeout() {
    if (now() - lastFrameTick > COMM_TIMEOUT_MS) {
        vehicleData->setCommunicationFault(true);
    } else {
        vehicleData->setCommunicationFault(false);
    }
}
```

Listing 4.3: Timeout-based link-health flag. Downstream UI reacts to the flag, never to UART timing directly.

4.6 Error Handling

Malformed packets and unrecognised or missing fields are silently discarded rather than crashing the parser or corrupting unrelated state. `VehicleData` independently tracks `framesReceived`, `invalidFrames`, and `checksumErrors` as running counters, surfaced on the Engineer Mode Communication page for field diagnosis. The `checksumErrors` counter remains permanently zero because the current telemetry protocol carries no checksum field; this is an identified gap discussed in Chapter 13.

5 Reactive Data Model

5.1 Design Rationale — Single VehicleData Object

VehicleData exposes every piece of vehicle state as Q_PROPERTY declarations. A UI developer working purely in QML needs only the property names on vehicleData and its singleton companions (Colors.qml, Typography.qml, Theme.qml). A backend engineer adding a new sensor adds one property to VehicleData.h, one setter to VehicleData.cpp, one token to TelemetryParser, and one QML binding at the display site. No QML file outside the relevant page requires modification.

Every screen in the application — Home, Diagnostics, and all of Engineer Mode — reads from the same VehicleData instance. No two views can ever display inconsistent state, because there is only one copy of each value.

5.2 IDataSource — The Strategy Pattern

Both SerialManager and TelemetrySimulator implement a single-method interface, start(). main.cpp constructs whichever implementation matches the current run mode and calls start() polymorphically. From that point on, VehicleData receives identical update calls regardless of whether the producer is a physical UART frame or a synthetic timer tick.

```
IDataSource* source;

/* 1 // pick the strategy once, at startup */
if (hardwareModeRequested)
    source = new SerialManager(port, vehicleData);
else
    source = new TelemetrySimulator(vehicleData);

/* 2 // from here on, VehicleData cannot distinguish which */
source->start();
```

Listing 5.1: main.cpp — polymorphic data-source selection via IDataSource.

This abstraction allows the entire UI and diagnostics stack to be exercised and demonstrated without any vehicle hardware attached. Every feature described in subsequent chapters was validated in simulation mode.

5.3 TelemetrySimulator — Internal Tick Loop

TelemetrySimulator maintains an internal SimulationState structure that mirrors the shape of VehicleData, and advances it on a QTimer tick to emulate a moving, thermally-evolving vehicle rather than static or random values. Controller and keyboard input maps onto this state the same way accelerator and brake pedals map onto the VCU's throttle_percent.

```
/* runs on a fixed-period QTimer (100 ms default) */

/* 1 // read the active input device */
throttle = controller.readAxis(THROTTLE_AXIS); /* or keyboard key */

/* 2 // advance the simulated vehicle through a simplified physical model */
state.speed      = accelerateToward(state.speed, throttle);
state.motorTemp  += thermalLoadFrom(state.speed, dt);
state.batteryPercent -= drainFrom(state.speed, dt);
state.rpm        = rpmFromSpeed(state.speed, state.driveMode);

/* 3 // push through the same setters TelemetryParser would use */
vehicleData->setSpeed(state.speed);
vehicleData->setMotorTemp(state.motorTemp);
vehicleData->setBatteryPercent(state.batteryPercent);
vehicleData->setRpm(state.rpm);
```

Listing 5.2: TelemetrySimulator per-tick update, converging on the same VehicleData interface as real telemetry.

Because the simulator and the real parser converge on the same interface, the simulator functions as an informal integration test: any UI bug reproducible in simulation will reproduce identically against real telemetry, since the UI cannot distinguish the two sources.

5.4 Drive Mode and Indicator Simulation

The PlayStation 3 controller's shoulder buttons toggle the simulated drive mode between ECO, CITY, and SPORT. The D-pad selects the simulated PRND gear state. Both are reflected immediately in `vehicleData.driveMode` and `vehicleData.gearState`, exercising the exact same QML bindings used in hardware mode. Turn indicator flags are toggled by the controller's face buttons, causing the indicator arrows to animate identically to a hardware-driven indicator event.

6 Driver Mode

The driver-facing application presents four pages reachable from a persistent bottom navigation bar. A fixed header bar at the top of every page displays the system status — connection indicator ("LIVE TELEMETRY" or a fault badge), system time, active drive mode, and battery percentage — so the driver has these at a glance regardless of the active page.

6.1 Home — Primary Driving View

The Home page is the first page displayed at application start. It is structured into five visual panels arranged to avoid the driver's eyes sweeping across the screen: the most time-critical information (speed, RPM, gear) occupies the largest left-side panel, while supplementary data occupies smaller panels to the right and bottom.

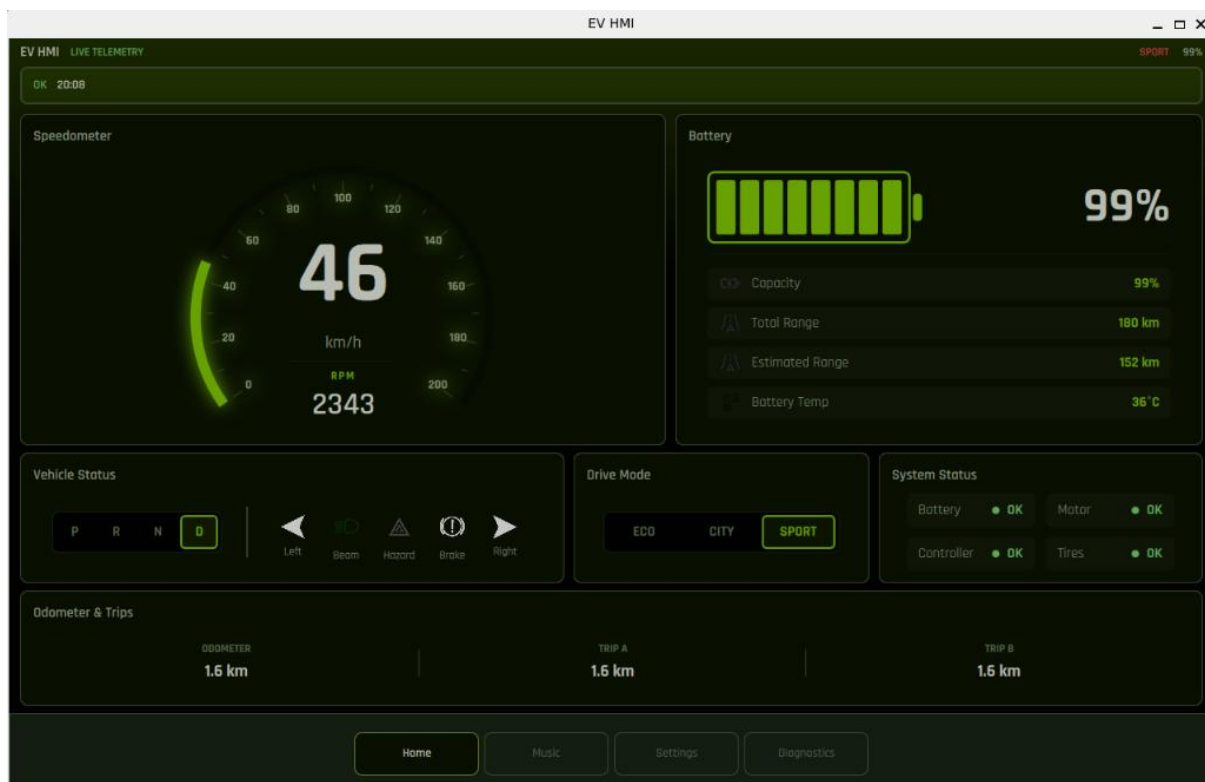


Figure 6.1: Home dashboard — analog speedometer with digital RPM readout, battery panel, vehicle status strip, drive-mode selector, system-status summary, and odometer/trip counters.

The Speedometer panel presents a circular analog gauge with a sweeping arc rendered in the current theme accent colour, a large digital speed readout in km/h or mph according to the active unit setting, and a digital RPM readout below. The arc rotation is driven by a QML binding on `vehicleData.speed`; the digital readout is driven by the same property, so both always agree.

The Battery panel shows a segmented battery icon whose fill level tracks `vehicleData.batteryPercent`, alongside numerical values for capacity (%), total range, estimated range, and battery temperature. The estimated range is computed by multiplying the current battery percentage against the known total range, both sourced from `VehicleData`.

The Vehicle Status strip sits below the speedometer and contains: the PRND gear indicator (the active gear highlighted in the theme accent colour), animated left and right turn-indicator arrows, a beam icon, a hazard icon, and a brake indicator. Every icon's active/inactive state is bound to the corresponding `VehicleData` flag. To the right of the gear/indicator row is the Drive Mode selector displaying ECO, CITY, and SPORT with the active mode highlighted.

The System Status panel provides a compact subsystem health summary: Battery, Motor, Controller, and Tyres, each with an OK / FAULT indicator dot driven by WarningManager. The Odometer and Trips panel at the bottom shows cumulative odometer distance and two resettable trip counters (Trip A, Trip B), integrated locally from vehicleData.speed.

6.2 Music — Media Playback

The Music page provides access to both local audio files and Spotify through a single tabbed interface. The active tab is selected by a toggle at the top left; the remaining layout is identical regardless of which backend is serving the content.

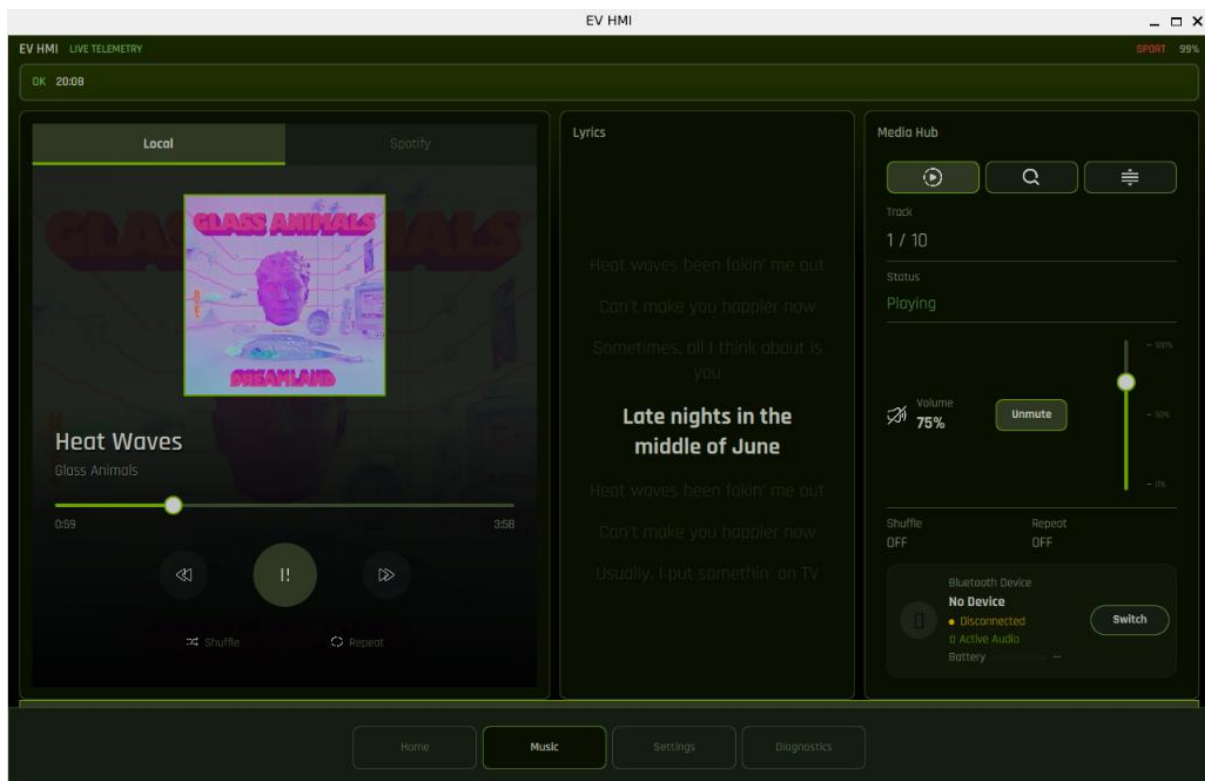


Figure 6.2: Music page — local playback active, showing album art, track progress, transport controls, synchronised lyrics panel, and the Media Hub with Bluetooth pairing status.

The left column displays album artwork, track title, artist name, a scrubbable progress bar with current/total time, and transport controls (previous, play/pause, next) with shuffle and repeat toggles. All controls are bound to the currently active media backend — LocalMusicPlayer or SpotifyAPIManager — through a shared property contract, so the QML is backend-agnostic.

The centre column is the Lyrics panel, which presents synchronised lyrics with the current line highlighted and surrounding lines at reduced opacity and smaller scale. As playback advances, the highlighted line scrolls to remain centred, creating a teleprompter-style effect. Lyrics are sourced from an embedded lyrics engine that matches playback position against a timestamped lyric file.

The right column is the Media Hub, which presents transport status (track index, playback state), a vertical volume slider with a mute toggle, shuffle and repeat state, and a Bluetooth panel showing the connected device name, connection status, active-audio flag, and device battery level, alongside a Switch button to initiate pairing.

On the Spotify tab, the same layout is used but the left column additionally presents a search interface backed by the Spotify Web API, and album art is fetched remotely rather than from local files.

6.3 Diagnostics — Driver Health Summary

The Diagnostics page provides a non-technical, at-a-glance health overview aimed at the driver rather than a technician. It answers the question "is everything working?" without exposing the level of detail that belongs in Engineer Mode.



Figure 6.3: Driver-facing Diagnostics page — vehicle topology diagram with live temperatures, overall health score, active-warning banner, and subsystem status tiles for Battery, Powertrain, Thermal, and Communication.

The Vehicle Overview panel on the left presents a schematic top-view of the vehicle with labelled thermal readings at each subsystem node (Motor, Battery Pack, Controller). Pack Voltage and Pack Current are displayed alongside the topology diagram, giving the driver an indication of where thermal load is concentrated without requiring navigation to Engineer Mode.

The Vehicle Health panel provides an Overall Score (0–100%) and a Status label (HEALTHY / DEGRADED / CRITICAL) derived from WarningManager's aggregate evaluation of all active flags. Individual subsystem rows (Battery Pack, Motor, Controller, Comms) each show an OK / FAULT indicator.

The Active Warnings panel shows either a "NO ACTIVE WARNINGS" confirmation or a stack of active warning banners. A Last Fault (Historical) row below it shows the most recently resolved fault with its timestamp and a Resolved badge.

The lower row of compact tiles provides specific numerical readings: Battery (SoC %, range, state of health), Powertrain (motor power in kW), Thermal (maximum temperature across all nodes), and Communication (CONNECTED / DISCONNECTED, stability indicator). A Powertrain Data panel and Fault History panel with an Acknowledge button round out the page.

6.4 Settings — Personalisation Controls

The Settings page is organised into four functional panels: Modes, Display, Volume, and System. Every setting takes effect immediately across the entire application — there is no Apply button and no page restart — because all preferences are published through QML singletons that every component is already bound to.

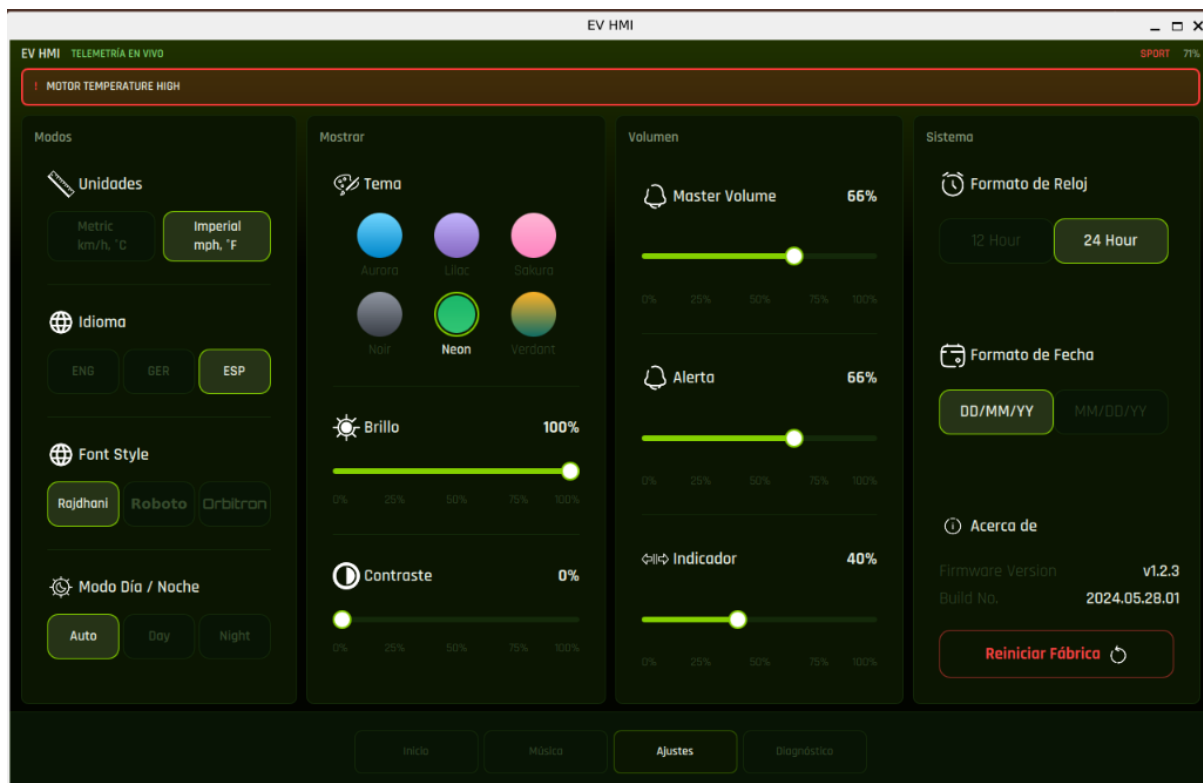


Figure 6.4: Settings page (Neon theme, Night mode) — Modes panel (left), Display panel with theme swatches and Brightness/Contrast sliders (centre-left), Volume panel (centre-right), and System panel with Clock Format, Date Format, About, and Factory Reset (right).

The Modes panel groups controls that affect how information is presented: a Metric/Imperial units toggle, a language selector (ENG / GER / ESP) that triggers a runtime QTranslator reload, a font-style selector (Rajdhani / Roboto / Orbitron), and a Day/Night/Auto mode selector. In Auto mode the HMI evaluates the system clock against configured dawn/dusk times and switches the palette accordingly.

The Display panel exposes the six named accent themes: Aurora, Lilac, Sakura, Noir, Neon, and Verdant. Each is represented by a colour swatch. Selecting a theme updates Colors.qml's active theme anchor, from which all surface, panel, border, and text colours are re-derived within one frame. Brightness and Contrast sliders operate as independent multipliers on the colour pipeline.

The Volume panel provides three independent sliders: Master Volume, Alert (notification and warning sound level), and Indicator (turn-signal click level). The three channels are kept separate so that silencing the indicator click does not affect music volume. All three values are persisted across sessions through Qt's QSettings mechanism.

The System panel controls time and date presentation and provides system information including firmware version and build number. The Factory Reset button, styled in a distinct warning colour, clears all user preferences and restores defaults after an explicit confirmation gesture.

7 Media Framework

7.1 Architecture

The media subsystem is deliberately isolated from the vehicle telemetry subsystem. Vehicle data never populates any media backend class, and music metadata is never added to `VehicleData`. This separation is enforced at the class boundary: the Music page QML is bound to `LocalMusicPlayer` or `SpotifyAPIManager` properties, never to `VehicleData` properties, and vice versa.

[Figure 7.1 — Media architecture diagram]

Figure 7.1: Media framework architecture. Spotify API, Bluetooth, and local files each have a dedicated backend class. QML is backend-agnostic through a shared property contract.

The three-class media split (`LocalMusicPlayer` / `SpotifyAPIManager` / `BluetoothManager`) is a deliberate ownership decision: each owns exactly one transport, and QML is never permitted to reach past the designated object to talk to `QMediaPlayer` or an OAuth token directly.

7.2 LocalMusicPlayer

`LocalMusicPlayer` wraps Qt's `QMediaPlayer` to expose local audio file playback to QML. Exposed properties include: `currentTrack`, `currentArtist`, `currentAlbum`, `albumArtUrl`, `playbackPosition`, `totalDuration`, `isPlaying`, `shuffleEnabled`, `repeatMode`, and `volume`. These property names are shared with `SpotifyAPIManager`, which is what allows the Music page QML to be backend-agnostic.

Playback controls (play, pause, stop, next, previous, seek) are exposed as invocable C++ slots. The playlist is backed by a `QMediaPlaylist` that supports both sequential and shuffled ordering. Album artwork is loaded asynchronously via `QImageProvider` to avoid blocking the UI thread.

7.3 SpotifyAPIManager

`SpotifyAPIManager` provides the same property contract as `LocalMusicPlayer` for Spotify-sourced content. It communicates with the Spotify Web API over HTTPS using Qt Network and authenticates via OAuth 2.0. Search queries entered on the Spotify tab are forwarded to the Spotify search endpoint, and results are presented as a selectable track list. Album art is fetched remotely via URL bindings in QML.

The OAuth token lifecycle (initial authorisation, token refresh on expiry) is managed within `SpotifyAPIManager`. The QML layer observes only the resulting playback state properties; authentication complexity is encapsulated entirely in the backend class.

7.4 BluetoothManager / DummyBluetoothManager

`BluetoothManager` manages Bluetooth audio device discovery, pairing, connection state, and active-audio routing using Qt Bluetooth. The Media Hub panel on the Music page binds to `BluetoothManager` properties: `connectedDeviceName`, `connectionStatus`, `activeAudio`, and `deviceBattery`.

`DummyBluetoothManager` is a no-op stand-in that exposes the same property interface with static placeholder values. When Bluetooth hardware is unavailable, the application substitutes `DummyBluetoothManager` at startup through the same conditional construction pattern used for `IDataSource`. Full Bluetooth hardware validation against a real device was not completed at the time of writing; this is noted as a limitation in Chapter 13.

7.5 Lyrics Engine

The lyrics display on the Music page is backed by an embedded lyrics engine that parses a timestamped lyric file (LRC format) and maintains a `currentLyricIndex` property updated against playback position.

The QML Lyrics panel binds a `ListView` to the lyric model, with the current index determining highlight state and scroll position, creating a teleprompter-style centred scroll as playback advances.

8 Theme Engine

8.1 Design Approach

The theme engine constructs all colours from two hue anchors per theme: a quiet `surfaceHue` for backgrounds, panels, and borders, and a vivid `accentHue` used sparingly for active indicators and highlighted text. All six themes share an identical lightness ramp across elevation levels — a raised card reads exactly as much brighter than its background in every theme. This systematic approach means that adding a seventh theme requires specifying only two hue values; the complete colour set is derived automatically.

Theme	Surface Hue Character	Accent Hue Character
Aurora	Cool blue-green	Teal / cyan
Lilac	Muted violet	Soft purple
Sakura	Warm rose	Pink / blush
Noir	Near-achromatic grey	High-contrast white
Neon	Deep dark base	Electric green
Verdant	Earthy green	Mid green

Table 8.1: Theme hue character definitions. Two anchors (`surfaceHue`, `accentHue`) generate the complete colour set for each theme.

8.2 Runtime Switching

Theme selection writes to a property in `Colors.qml`. Because `Colors.qml` is a QML singleton, every component that references its color tokens is already subscribed to its property-change signals. A theme switch propagates to every visible element within one frame, with no restart and no re-initialisation of the QML engine. The same mechanism applies to day/night mode switching: `Colors.qml` re-evaluates all derived colours from a single `isDarkMode` boolean.

8.3 Hue-Locked Status Colours

Two drive-mode accent colours — ECO green and SPORT amber — are hue-locked across all themes. Regardless of the active theme, the ECO indicator is always rendered in the same green, and the SPORT indicator in the same amber. This design decision ensures that safety-relevant status colours retain their semantic meaning even when the user selects an unusual theme combination.

8.4 Brightness and Contrast Controls

The Display panel's Brightness and Contrast sliders operate as independent multipliers applied to the colour pipeline after theme derivation. Brightness scales the luminance of all surface and background colours. Contrast adjusts the luminance gap between surface and panel layers, affecting the visual elevation hierarchy. These two controls allow fine-grained adaptation to ambient lighting conditions without changing the theme anchor hues.

8.5 Typography and Scaling

`Typography.qml` exposes font family and size scale constants referenced by every text element. Three font choices are available: Rajdhani (default, condensed letterforms optimised for numerical telemetry data), Roboto (neutral, legibility-optimised), and Orbitron (stylistic, techno character). Font selection writes to a `Typography.qml` property, causing all bound text elements to update

immediately. A global scale factor in Theme.qml multiplies every spacing, radius, and font-size constant, allowing the UI to adapt to different display densities.

9 Engineer Mode

Engineer Mode is an intentionally hidden diagnostic surface accessible only through a long-press gesture on the Diagnostics page. The separation between Driver Mode and Engineer Mode is deliberate: the information presented in Engineer Mode — raw UART frame statistics, fault history with machine-readable codes, per-subsystem thermal trend data, live telemetry parameter dumps, and simulator controls — is meaningful to a technician or developer but would add unnecessary complexity to the driver's daily experience.

Architecturally, Engineer Mode reads from exactly the same VehicleData instance as Driver Mode. It is a more detailed view over the same reactive model, not a separate data pipeline. The visual language shifts to a high-contrast, dense-information layout with monospace typefaces for numerical data, appropriate for a workshop environment.

9.1 Overview Page

The Overview page replicates the Diagnostics page's vehicle topology diagram and health score, adds a Quick Statistics panel showing key numerical values (speed, RPM, battery voltage, motor power, temperatures), and displays an active-warning summary and fault history entry. The intent is to establish at a glance whether the vehicle is in a serviceable state before navigating to detailed sub-pages.

9.2 Live Telemetry Page

The Telemetry page presents every VehicleData property as a live numerical readout, organised into parameter groups (Powertrain, Thermal, Battery, Communication). Values update at the telemetry cadence. The most recently decoded frame is also displayed as its raw key=value text, giving a technician immediate visibility into what the VCU is actually transmitting without requiring a separate serial monitor.



Figure 9.1: Engineer Mode — Live Telemetry page showing grouped parameter readouts and raw frame display.

9.3 Thermal Page

The Thermal page shows current temperatures for the Motor, Battery Pack, and Controller alongside horizontal bar gauges colour-coded by normal/elevated/high classification. Trend graphs plot temperature over time for each subsystem from a rolling buffer of timestamped readings maintained in memory. Warning thresholds and the current temperature-warning flag status are shown, allowing a technician to determine how close each subsystem is to triggering a fault condition.

9.4 Powertrain Page

The Powertrain page presents the full powertrain electrical data: Battery Voltage, Battery Current, Motor Power, Regen Level, and Motor RPM, each as a live numerical readout. Trend graphs for Voltage, Current, and Power show their evolution over the current session.

9.5 Communication Page

The Communication page is the diagnostic tool for the serial link itself. It displays current UART connection status (port name, baud rate), parser statistics (frames received, invalid frames, checksum errors), frame quality as a percentage, protocol version, and a connection-health history log. An Export Logs button invokes TelemetryLogger to write the current session's log to a timestamped CSV file.

Displayed Metric	Source	Engineering Use
Frames received	TelemetryParser counter	Verify link is running at expected 10 Hz cadence.
Invalid frames	TelemetryParser counter	Localise parser rejection vs UART noise.
Checksum errors	TelemetryParser counter (currently unpopulated)	Requires protocol checksum addition.
Frame quality %	Derived: $(\text{valid}/\text{total}) \times 100$	Quick link-health indicator.
Communication fault flag	SerialManager watchdog	True when > 1000 ms without valid frame.

Table 9.1: Communication page displayed metrics and their engineering utility.

9.6 Fault Manager Page

The Fault Manager page is the most detailed diagnostic tool in Engineer Mode. An active fault entry shows: priority level (CRITICAL / WARNING / INFO), a machine-readable fault code (e.g. MOT_TEMP_HIGH), a human-readable description, the timestamp at which the fault was raised, the current sensor reading vs. the threshold that triggered it, and a short list of possible root causes. This mirrors production automotive DTC (Diagnostic Trouble Code) tooling at a scope appropriate for this project.

A fault history table lists all faults logged during the current session. Running fault statistics (total raised, total acknowledged, total resolved) are displayed. Acknowledge, Clear Faults, and Run Self Test controls are available. When running in simulation mode, an additional Simulator Controls panel allows direct injection of specific sensor values — forcing simulated motor temperature to 68 °C, for example, to verify that the thermal warning pathway fires correctly end-to-end.

10 Telemetry Simulation

10.1 Purpose and Scope

The TelemetrySimulator serves three distinct purposes during the development lifecycle: it provides a functional UI stack exercisable without physical VCU hardware; it acts as an informal integration test by exercising the full QML binding chain end-to-end; and through the Engineer Mode Simulator Controls panel, it enables deliberate fault injection for validating warning and fault pathways.

10.2 Physical Modelling Fidelity

The simulator is not a high-fidelity vehicle dynamics model. It uses simplified equations of motion sufficient to produce plausible, causally-consistent telemetry: speed changes in response to throttle input through a first-order lag, motor temperature rises with speed-derived thermal load and decays toward ambient when stationary, battery percentage depletes as a function of motor power, and RPM is derived from speed and the current drive mode's gear ratio approximation.

This level of fidelity is sufficient for UI validation: it produces continuous, non-trivial value changes that exercise all animated elements, it can sustain values in the warning and fault regions long enough for threshold crossings to be observed and acknowledged, and it responds to user input in a way that approximates expected driving behaviour.

10.3 Input Devices

The simulator accepts input from a PlayStation 3 controller via USB HID or from a USB keyboard. Controller axes map to throttle and brake; shoulder buttons change drive mode; the D-pad selects gear state (PRND); face buttons toggle turn indicators, hazard, and beam states. Keyboard mappings provide equivalent functionality on machines without a connected controller. The input abstraction is contained within TelemetrySimulator, so adding a new input device requires only changes to that class.

10.4 Fault Injection

The Engineer Mode Simulator Controls panel exposes direct override of any simulated sensor value. This bypasses the physics model and sets the value immediately, allowing a developer to verify:

- That motor temperature above `MOTOR_TEMP_WARNING_C` raises the correct `WarningManager` flag.
- That the warning flag propagates to the Home page System Status panel and the Diagnostics page Active Warnings panel simultaneously.
- That the fault is logged correctly by `TelemetryLogger` with the correct timestamp and reading.
- That the Acknowledge and Clear Faults controls on the Fault Manager page operate as expected.

This direct-injection capability makes Engineer Mode's simulator controls the fastest path to verifying fault handling without waiting for the physical system to reach the relevant state.

11 Logging Framework

11.1 TelemetryLogger

The TelemetryLogger class provides persistent, exportable records of telemetry data and fault events. Its primary purpose is to enable post-session analysis and to create a ground-truth dataset that can be compared against the STM32's own records — for example, to verify that all charge/discharge events recorded by the VCU are reflected in the HMI's received frame stream without gaps.

11.2 Telemetry Export

At the end of a session, or on demand from the Engineer Mode Communication page, TelemetryLogger writes a CSV file containing every received telemetry frame with a monotonic timestamp. Files are named with the session start time in the format YYYY-MM-DD_HH-MM-SS_telemetry.csv, ensuring successive sessions never overwrite each other.

```
timestamp_ms, SPD, RPM, BAT, MT, BT, RNG, PWR, MODE, GEAR
0, 54, 2800, 87, 41, 35, 120, 4.3, ECO, D
100, 55, 2820, 87, 41, 35, 120, 4.4, ECO, D
200, 56, 2840, 86, 42, 35, 119, 4.5, ECO, D
...
```

Listing 11.1: Telemetry CSV row format. timestamp_ms is elapsed time since session start. Fields absent from a given frame are recorded as empty cells.

This format is directly importable into Python/pandas, MATLAB, or Microsoft Excel for offline analysis. Time-aligned comparison of two quantities requires no additional processing beyond selecting the relevant columns.

11.3 Fault and Warning Export

TelemetryLogger maintains a separate log for warning and fault events, written to YYYY-MM-DD_HH-MM-SS_faults.csv. Each entry records the event type, fault code, session-relative timestamp, and the sensor reading that triggered it.

```
timestamp_ms, event_type, fault_code, reading, threshold
32400, WARNING_RAISED, MOT_TEMP_HIGH, 56, 55
35700, WARNING_CLEARED, MOT_TEMP_HIGH, 53, 55
87200, FAULT_RAISED, BAT_LOW, 14, 15
```

Listing 11.2: Fault log CSV row format.

11.4 Diagnostic Utility

The exported logs serve three primary purposes. First, they provide a ground truth for debugging intermittent communication issues: a gap in the timestamp sequence at a nominally 100 ms cadence indicates dropped frames, which combined with the Communication page's invalidFrames counter can localise whether the issue is UART line noise, parser rejection, or OS scheduling latency. Second, they enable post-session performance analysis through direct plotting of speed and motor power against session time. Third, the fault log supports root-cause analysis for intermittent fault conditions that are not visible from the Fault Manager page's snapshot-in-time view.

12 Design Evaluation

12.1 Architectural Advantages

Modularity and separation of concerns. The strict ownership boundaries described throughout this report mean that any one component can be replaced without modifying the others. TelemetrySimulator was substituted for SerialManager during the entire development and validation phase without modifying a single QML file. The three media classes share an identical property contract so the Music page is transport-agnostic. A CAN Bus data source could be introduced as a fourth IDataSource implementation without altering any existing component.

Scalability of the reactive model. VehicleData's Q_PROPERTY pattern scales to any number of properties at near-zero UI cost. Adding a new sensor value requires one property declaration, one setter, one token in TelemetryParser, and one QML binding. The property-binding engine propagates the change to every bound element automatically.

Reusable component library. The QML component library (BaseCard, speedometer, battery icon, indicator strip, media controls) is parameterised rather than hardcoded, making each component reusable across pages and adaptable to a different screen size through the single Theme.qml scale factor. The theme engine generates twelve visual configurations from two hue anchors per theme and one shared lightness ramp.

Embedded deployment readiness. The application runs on a consumer-grade SBC under a standard Linux userspace, which is a realistic automotive infotainment architecture. Qt's hardware-accelerated rendering pipeline uses OpenGL ES, available on the Raspberry Pi's VideoCore GPU, so UI compositing does not burden the CPU. UART is handled by QSerialPort, a mature cross-platform abstraction that requires no changes if the target hardware is changed.

Maintainability. The codebase follows a consistent naming convention — Q_PROPERTY names match telemetry token names where possible — and every QML colour, font, and spacing value references a named token from Colors.qml / Typography.qml / Theme.qml rather than being hardcoded.

12.2 Engineering Rationale for Key Decisions

Decision	Rationale
Key=value telemetry format over fixed CSV	Parser resilience: unknown or malformed tokens are skipped without corrupting remaining fields. Either side can add tokens independently.
Single VehicleData instance shared by all views	Enforces consistency: no two views can display different values for the same property.
IDataSource strategy pattern	Allows complete UI validation without hardware, and future data sources (CAN Bus) require no QML changes.
Singleton QML theme/typography objects	Design tokens propagate to all components without prop drilling; theme changes are instantaneous and global.
Separate media backend classes per transport	Each class owns one resource domain; QML is agnostic about which transport is active.
WarningManager as separate C++ class	Threshold values are centralised; a threshold change requires one code edit rather than a grep across QML files.
TelemetryLogger CSV format	Directly importable in standard analysis tools (pandas, MATLAB, Excel) with no transformation step.

Table 12.1: Key design decisions and engineering justifications.

13 Limitations

The following limitations were identified during implementation. They are documented without mitigation to provide an accurate characterisation of the system as built.

Limitation	Impact	Mitigation Path
No checksum in telemetry protocol	checksumErrors counter permanently zero; corrupt frames indistinguishable from short frames	Add CRC-8 or XOR checksum to frame format on both STM32 and parser side
Bluetooth audio not hardware-validated	BluetoothManager pairing and audio-routing flows not exercised against a real device	Hardware validation against Raspberry Pi 5 Bluetooth subsystem required
Incomplete localisation coverage	Hard-coded strings not wrapped in qsTr() will not switch when language selector changes	Full audit of all QML string literals; wrap in qsTr() consistently
Engineer Mode access control absent	Any user knowing the long-press gesture can access Engineer Mode	PIN-gate or physical-key requirement needed for production deployment
Documented protocol (CSV) differs from implemented protocol (key=value)	Internal documentation inconsistency	Update docs/communicationprotocol.md to reflect the implemented key=value format
Odometer integrated locally, not transmitted	Odometer resets on application restart; does not persist between power cycles	Transmit cumulative odometer from VCU, or persist locally across sessions
No checksum validation between protocol documentation and implementation in CI	Documentation drift can recur undetected	Add automated CI diff check between protocol spec and parser token list

Table 13.1: Identified limitations, impact, and mitigation paths.

14 Future Scope

The following extensions are grounded in the implemented architecture and represent realistic next steps rather than speculative additions. Each is anchored to a specific existing component that provides the extension point.

14.1 CAN Bus Integration

The most impactful near-term extension would be replacing the UART link with a CAN Bus interface. CAN Bus is the industry-standard vehicle network interface and would allow the HMI to receive from multiple VCUs on the same bus without dedicated wiring per device. Architecturally, a CAN Bus implementation would be a new `IDataSource` implementation (`CANBusManager`) inserted in place of `SerialManager`; `VehicleData` and all QML components would require no modification. The STM32F446RE in the companion report has a `bxCAN` peripheral that is currently unused; enabling it on both sides is a natural companion to this work.

14.2 CAN FD Support

CAN FD (Flexible Data-rate) extends the classic CAN protocol to 64-byte payloads at higher bit rates, which would allow a single CAN FD frame to carry the entire telemetry record currently split across multiple UART tokens. The STM32F446RE does not have CAN FD hardware; this extension would require either a CAN FD controller IC or a migration to an STM32G4-series MCU, which supports CAN FD natively.

14.3 OTA Firmware Updates

The Raspberry Pi is a networked device capable of downloading software updates. A Qt-based OTA update client could allow the HMI application to be updated over Wi-Fi without requiring physical access to the unit. The STM32 firmware could additionally be updated over the existing UART link using the STM32 bootloader's UART DFU mode, enabling field updates of the VCU without hardware intervention.

14.4 Android Auto and Apple CarPlay

Integration with Android Auto or Apple CarPlay would allow the driver's smartphone to project navigation, phone, and media applications onto the HMI display. Both protocols are supported on Linux hosts. CarPlay requires Apple's MFi licensing; Android Auto is open-source via the Open Automotive Alliance. The Music page's existing transport abstraction provides a natural extension point for an `AndroidAutoMediaSource` implementation.

14.5 Voice Assistant Integration

A local wake-word engine (e.g. PocketSphinx or a quantised Whisper model running on-device) would allow hands-free control of the HMI. The Qt multimedia stack provides the audio capture path; the voice pipeline would emit commands as simulated user interactions against the same QML property bindings already used by touch input, requiring no new UI code.

14.6 Cloud Diagnostics and Fleet Monitoring

`TelemetryLogger`'s CSV output is designed for local analysis. A natural extension is a cloud backend that ingests timestamped telemetry sessions via MQTT or HTTPS, stores them in a time-series database, and exposes a web dashboard for fleet-level analysis. The Raspberry Pi's network connectivity makes this straightforward; `TelemetryLogger` would be extended to publish to an MQTT broker in addition to writing the local CSV.

14.7 Protocol Hardening

Adding a CRC-8 or simple XOR checksum to the UART frame format is a low-effort, high-value improvement that would populate the checksumErrors counter already surfaced on the Communication page. Synchronising the protocol documentation with the implemented key=value format and adding automated CI checks would prevent the documentation drift identified in Chapter 13 from recurring.

14.8 ADAS Integration

The existing warning infrastructure in WarningManager and the Active Warnings panel in both Driver and Engineer Mode provide a natural extension point for ADAS (Advanced Driver Assistance System) alerts — lane departure, collision proximity, blind-spot warnings — which are semantically identical to the existing over-temperature and low-battery warnings from the HMI's perspective. Adding ADAS alert sources would require only new WarningManager threshold evaluators and new fault codes; no UI structural changes would be required.

15 Conclusion

This report documents the driver- and engineer-facing HMI application that consumes the telemetry pipeline built in the companion VCU volume. Three architectural properties distinguish the implementation:

- A single reactive C++ data model (VehicleData) shared identically by the driver dashboard, the driver-facing Diagnostics page, and all six Engineer Mode pages.
- A hardware/simulation abstraction (IDataSource) that allows the entire application to be demonstrated, developed, and tested without the VCU attached.
- A six-theme, systematically-derived personalisation engine that generates twelve visual configurations from a small, principled set of design primitives.

Together with the companion VCU report, these two volumes cover the complete signal path from accelerator pedal and temperature sensor on the STM32 to animated speedometer arc and thermal trend display on the 10.1-inch touchscreen.

Chapter 12 identifies the design decisions that pay off — principally the modularity, the reactive model's scalability, and the component library's reusability. Chapter 13 provides a candid accounting of what is production-ready (reactive model, theme engine, simulator, media abstraction), what is implemented but not yet hardware-validated (Bluetooth audio), and what are known gaps requiring engineering effort before deployment (checksum, documentation synchronisation, Engineer Mode access control). Chapter 14 provides a prioritised and architecturally grounded view of realistic next steps toward a production-quality embedded HMI.

Appendix A VehicleData Property Table

Complete reactive property map, extending the summary in Chapter 1.

Property	Type	Unit	Notify Signal
speed	int	km/h	speedChanged()
rpm	int	RPM	rpmChanged()
batteryPercent	int	%	batteryPercentChanged()
motorTemp	int	°C	motorTempChanged()
batteryTemp	int	°C	batteryTempChanged()
controllerTemp	int	°C	controllerTempChanged()
range	int	km	rangeChanged()
motorPower	float	kW	motorPowerChanged()
regenLevel	int	0–5	regenLevelChanged()
driveMode	enum {ECO,CITY,SPORT}	—	driveModeChanged()
gearState	enum {P,R,N,D}	—	gearStateChanged()
packVoltage	float	V	packVoltageChanged()
packCurrent	float	A	packCurrentChanged()
stateOfHealth	int	%	stateOfHealthChanged()
odometer	float	km	odometerChanged()
tripDistanceA / tripDistanceB	float	km	tripDistanceAChanged() / tripDistanceBChanged()
leftIndicator / rightIndicator	bool	—	leftIndicatorChanged() / rightIndicatorChanged()
beamActive / hazardActive / brakeActive	bool	—	beamActiveChanged() / hazardActiveChanged() / brakeActiveChanged()
communicationFault	bool	—	communicationFaultChanged()
framesReceived	uint	count	framesReceivedChanged()
invalidFrames	uint	count	invalidFramesChanged()
checksumErrors	uint (unpopulated — §13)	count	checksumErrorsChanged()

Table A.1: Complete VehicleData Q_PROPERTY declaration list.

Appendix B Music API — Shared Property Contract

The following properties are exposed by both `LocalMusicPlayer` and `SpotifyAPIManager`. QML binds to these names; switching between backends requires no QML changes.

Property	Type	Description
<code>currentTrack</code>	string	Track title
<code>currentArtist</code>	string	Artist name
<code>currentAlbum</code>	string	Album name
<code>albumArtUrl</code>	url	Album artwork URL or local file path
<code>playbackPosition</code>	int	Current position in milliseconds
<code>totalDuration</code>	int	Track length in milliseconds
<code>isPlaying</code>	bool	True when playback is active
<code>shuffleEnabled</code>	bool	Shuffle state
<code>repeatMode</code>	enum {Off,Track,Queue}	Repeat mode
<code>volume</code>	int	Volume level 0–100
<code>trackIndex</code>	int	Current track position in playlist/queue
<code>queueCount</code>	int	Total tracks in active queue

Table B.1: Shared property contract between `LocalMusicPlayer` and `SpotifyAPIManager`.

Invokable slots exposed by both backends: `play()`, `pause()`, `stop()`, `next()`, `previous()`, `seekTo(int ms)`, `setVolume(int)`, `setShuffle(bool)`, `setRepeat(enum)`.

Appendix C Project Folder Structure

```
HMI-codebase/
├── CMakeLists.txt
├── main.cpp # App entry, IDataSource construction
├── backend/
│   ├── VehicleData.h / .cpp # Reactive data model
│   ├── SerialManager.h / .cpp # UART interface (hardware mode)
│   ├── TelemetryParser.h / .cpp # Key=value frame decoder
│   ├── TelemetrySimulator.h / .cpp # Software data source
│   ├── TelemetryLogger.h / .cpp # CSV export
│   ├── WarningManager.h / .cpp # Threshold evaluation
│   ├── LocalMusicPlayer.h / .cpp # QMediaPlayer wrapper
│   ├── SpotifyAPIManager.h / .cpp # Spotify OAuth + API
│   ├── BluetoothManager.h / .cpp # Qt Bluetooth
│   └── DummyBluetoothManager.h / .cpp
├── qml/
│   ├── main.qml # Root window and navigation
│   ├── singletons/
│   │   ├── Colors.qml # Theme colour tokens
│   │   ├── Typography.qml # Font family and size tokens
│   │   └── Theme.qml # Scale factor and layout constants
│   ├── pages/
│   │   ├── HomePage.qml
│   │   ├── MusicPage.qml
│   │   ├── DiagnosticsPage.qml
│   │   ├── SettingsPage.qml
│   │   └── engineer/
│   │       ├── EngineerOverview.qml
│   │       ├── TelemetryPage.qml
│   │       ├── ThermalPage.qml
│   │       ├── PowertrainPage.qml
│   │       ├── CommunicationPage.qml
│   │       └── FaultManagerPage.qml
│   └── components/ # Reusable QML components
│       ├── BaseCard.qml
│       ├── Speedometer.qml
│       ├── BatteryIcon.qml
│       ├── IndicatorStrip.qml
│       └── MediaControls.qml
├── assets/
│   ├── fonts/ # Rajdhani, Roboto, Orbitron
│   ├── music/ # Local audio files
│   └── lyrics/ # LRC timestamped lyric files
├── docs/
│   ├── communicationprotocol.md # Protocol spec (requires update – §13)
│   ├── architecture.md
│   ├── spotify.md
│   └── telemetryAPI.md
```

Listing C.1: Top-level project folder structure.

Appendix D References

- [1] Qt Documentation, "Qt QML and Qt Quick," The Qt Company. Available: <https://doc.qt.io/qt-6/qtqml-index.html>
- [2] Qt Documentation, "The Property System," The Qt Company. Available: <https://doc.qt.io/qt-6/properties.html>
- [3] Qt Documentation, "QSerialPort Class," The Qt Company. Available: <https://doc.qt.io/qt-6/qserialport.html>
- [4] Qt Documentation, "QMediaPlayer Class," The Qt Company. Available: <https://doc.qt.io/qt-6/qmediaplayer.html>
- [5] Björn Ottosson, "A perceptual colour space for image processing (Oklab)," 2020. Available: <https://bottosson.github.io/posts/oklab/>
- [6] Web Content Accessibility Guidelines (WCAG) 2.1, "Success Criterion 1.4.3: Contrast (Minimum)," W3C. Available: <https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html>
- [7] EV Vehicle Control Unit: Proof of Concept — Part 1 of this project, STM32F446RE VCU report, July 2026.
- [8] Spotify Developer Documentation, "Spotify Web API," Spotify AB. Available: <https://developer.spotify.com/documentation/web-api/>
- [9] STMicroelectronics, "STM32F446xC/E Datasheet," DS10693 Rev 8, 2023.
- [10] Raspberry Pi Ltd, "Raspberry Pi 5 Product Brief," 2023. Available: <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [11] IEEE, "IEEE Author Center," 2024. Available: <https://ieeeauthorcenter.ieee.org/>